

KHOA CÔNG NGHỆ THÔNG TIN



Công nghệ phần mềm

Chương III: Phân tích & Thiết kế kiến trúc hệ thống (Ph.2)



Nội dung

- Các chiến lược thiết kế
 - I. Phương pháp hướng chức năng (hướng cấu trúc hóa)
 - II. Phương pháp hướng đối tượng
 - III. Phương pháp hướng tương quan (Aspect-Oriented Architecture)



I. Phương pháp hướng chức năng

- Cách tiếp cận hướng chức năng (hướng cấu trúc)
- Phương pháp thiết kế cấu trúc hóa của Constantine.
 - Phân tích cấu trúc hóa
 - Các khái niệm
 - Các phương pháp thiết kế hệ thống theo cấu trúc hóa
 - Phương pháp dựa vào trung tâm biến đổi STS
 - Phương pháp phân chia giao dịch TR



Cách tiếp cận hướng chức năng

- Là cách tiếp cận truyền thống xoay quanh dữ liệu, chủ yếu quan tâm tới thông tin của hệ thống.
- Là phương pháp tốt để thiết kế ngân hàng dữ liệu và nắm bắt thông tin.
- Thách thức lớn với những hệ thống thường xuyên thay đổi vì khó thực thi những thay đổi trong nguyên tắc nghiệp vụ hay cách hoạt động của hệ thống.



Phương pháp thiết kế cấu trúc hóa

- Phương pháp thiết kế cấu trúc hóa của Constantine:
 - Bắt nguồn từ tính môđun, thiết kế từ trên-dưới, lập trình cấu trúc.
 - Còn xem như Phương pháp thiết kế hướng luồng dữ liệu (Data flow-oriented design)
 - Quy trình 6 bước: (1) tạo kiểu luồng thông tin; (2) chỉ ra biên của luồng; (3) ánh xạ DFD sang cấu trúc chương trình; (4) xác định phân cấp điều khiển; (5) tinh lọc cấu trúc; (6) chọn mô tả kiến trúc



Phương pháp thiết kế cấu trúc hóa

(1) Phân tích cấu trúc hóa

(2) Các khái niệm:

- Môđun và tham số
- Lưu đồ bong bóng và cấu trúc phân cấp

Lưu đồ bong bóng (Bubble chart)

Cấu trúc phân cấp (Hierarchical structured chart)

(3) Các phương pháp thiết kế hệ thống theo cấu trúc hóa



Phương pháp thiết kế cấu trúc hóa

1. Phân tích cấu trúc hóa :

- Xác định luồng dữ liệu
- Luồng tuyến tính thì theo phân chia STS
- Luồng phân nhánh thì theo phân chia TR



Phương pháp thiết kế cấu trúc hóa

2. Khái niệm

- **Mô đun:** Module A
 - Là một dãy các lệnh nhằm thực hiện một chức năng nào đó
 - Mô đun có tính độc lập tương đối, được soạn thảo và biên dịch độc lập
 - Môđun đã dịch có thể được môđun khác gọi tới.
 - Giao diện giữa các môđun thông qua các biến tham số (arguments). Tham số có 2 loại: tham số điều khiển (flag) và tham số mang tính dữ liệu thuần túy.



Phương pháp thiết kế cấu trúc hóa

- Thiết kế mô đun:

Dựa trên quan điểm “chia để trị”:

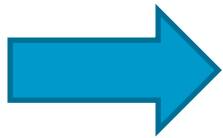
C: độ phức tạp

$$C(p1 + p2) > C(p1) + C(p2)$$

E: công sức thực hiện

$$E(p1 + p2) > E(p1) + E(p2)$$

- giảm độ phức tạp
- cục bộ, dễ sửa đổi
- có khả năng phát triển song song
- dễ sửa đổi, dễ hiểu nên dễ tái sử dụng



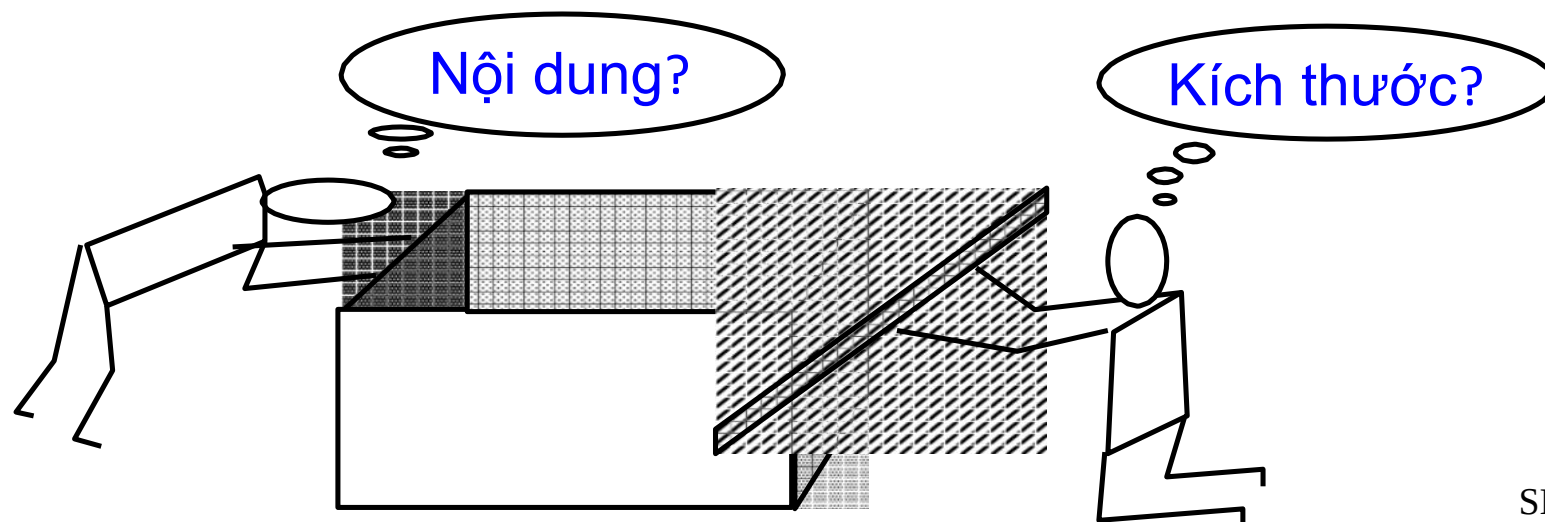
Là một khái niệm quan trọng xây dựng phần mềm!



Phương pháp thiết kế cấu trúc hóa

Thiết kế mô đun (tiếp)

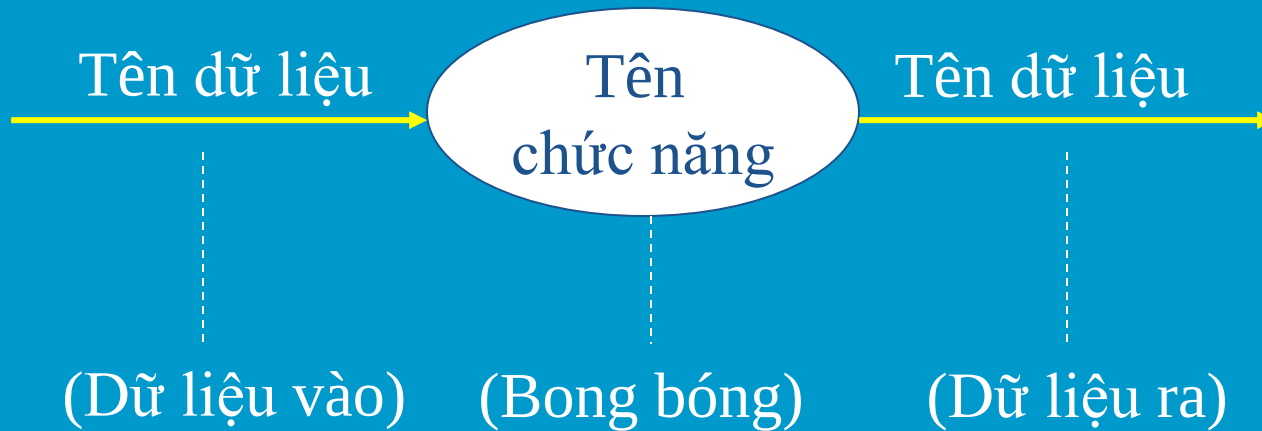
- Xác định kích cỡ mô đun – được quyết định dựa trên khái niệm độc lập chức năng: mỗi mô đun nên thực hiện 1 công việc → dễ hiểu, dễ sửa đổi, dễ tái sử dụng





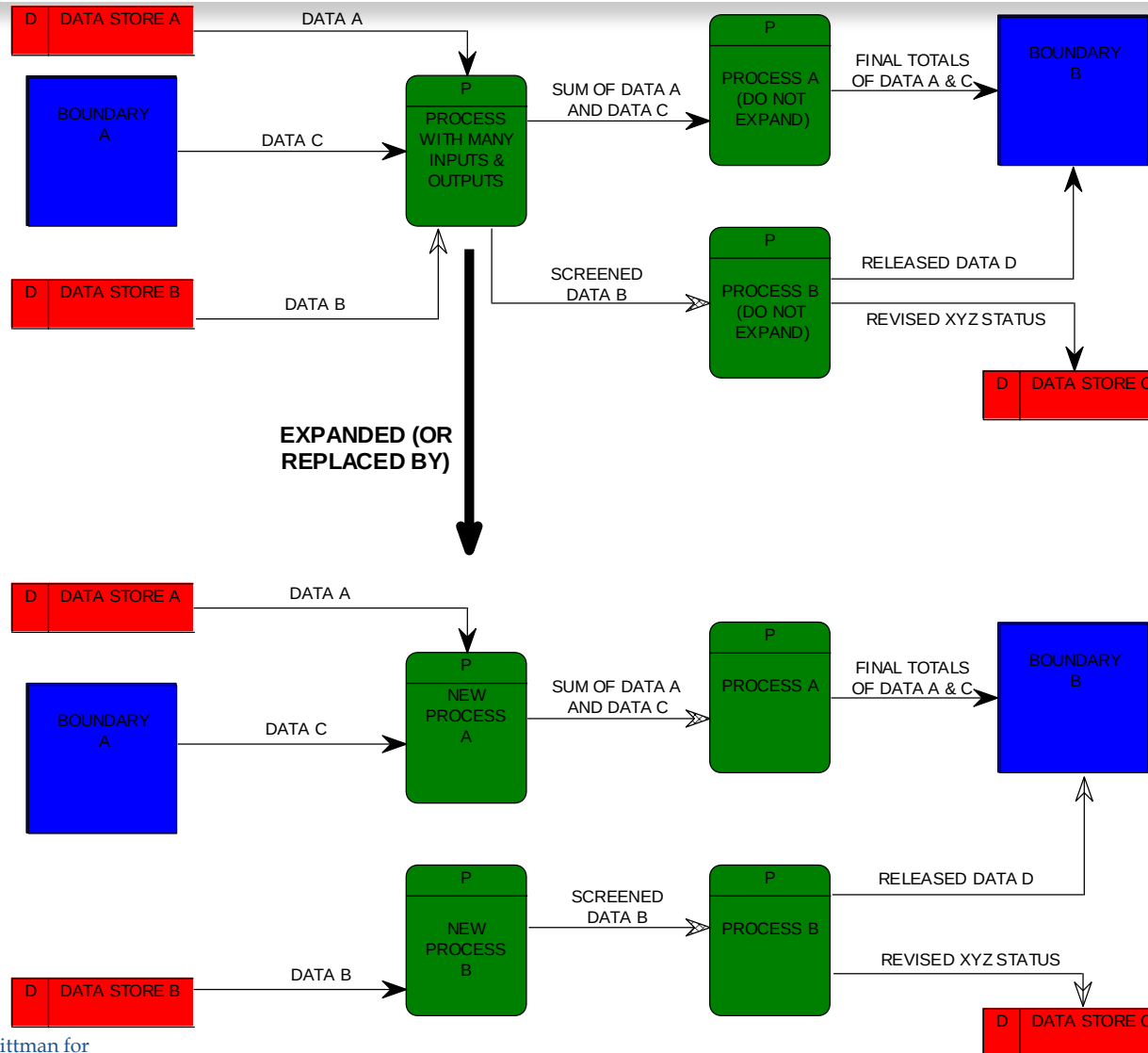
Lưu đồ bong bóng (Bubble chart)

- Biểu thị luồng xử lý dữ liệu
- Ký pháp





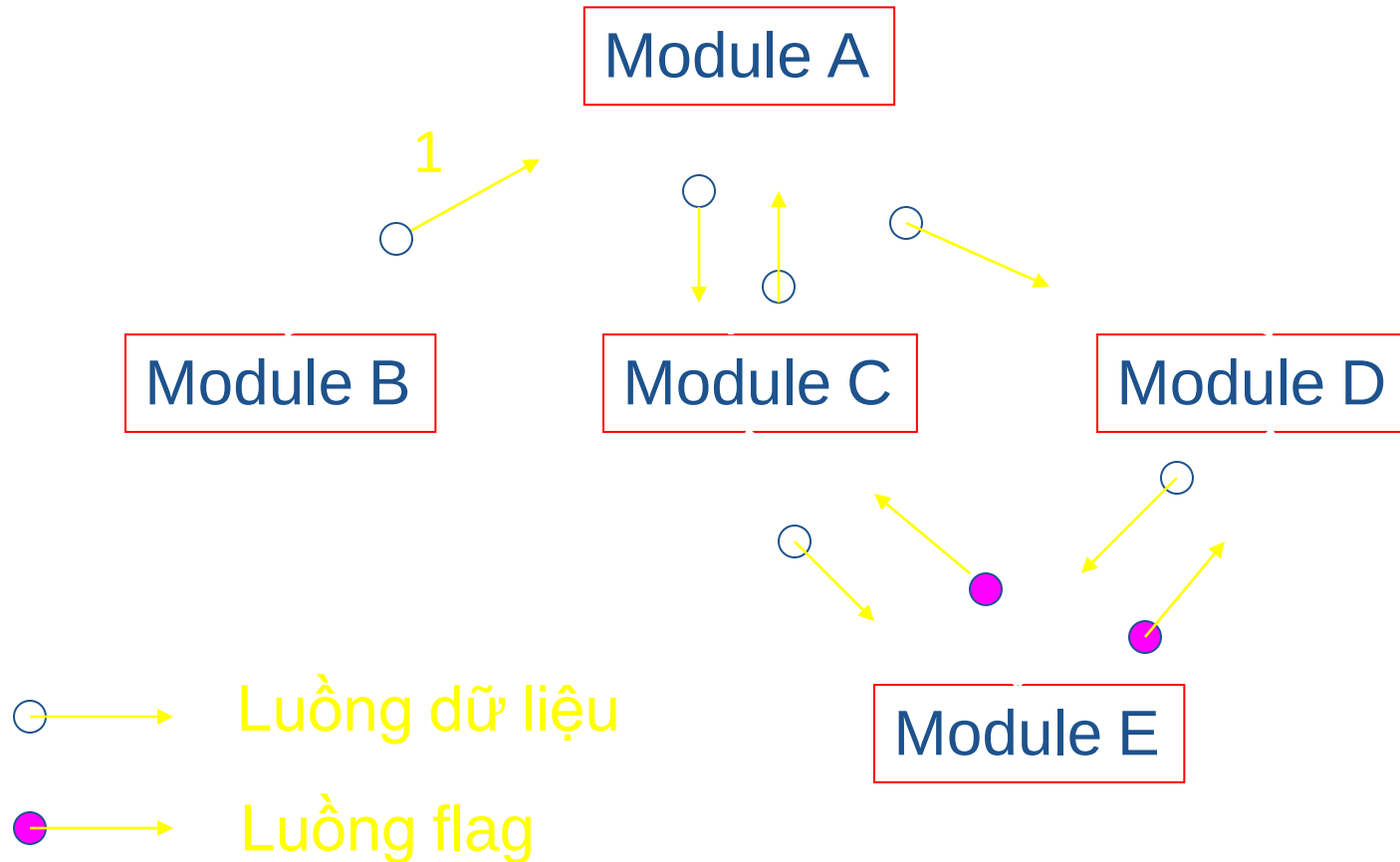
Luồng xử lý dữ liệu



Expanding a Multifunction Process on a DFD



Hierarchical structured chart





Phương pháp thiết kế cấu trúc hóa

3. Các phương pháp thiết kế hệ thống theo cấu trúc hóa

- Hai phương pháp phổ biến trong thiết kế hệ thống:
 - Phương pháp dựa vào trung tâm biến đổi STS (Source - Transform – Sink : Nguồn – Biến đổi – Hấp thụ).
 - Phương pháp dựa vào giao dịch TR (Transaction).



Biểu đồ luồng dữ liệu

- Phát triển một biểu đồ luồng dữ liệu là bước đầu tiên của thiết kế hướng chức năng. Nó chỉ ra cách biến đổi dữ liệu vào thành dữ liệu ra thông qua một dãy các phép biến đổi.
- Biểu đồ luồng dữ liệu là một phần hợp nhất của một số các phương pháp thiết kế và các công cụ CASE.

(Các ký pháp và cách biểu diễn sẽ được học trong môn phân tích và thiết kế hệ thống)



Phương pháp thiết kế cấu trúc hóa

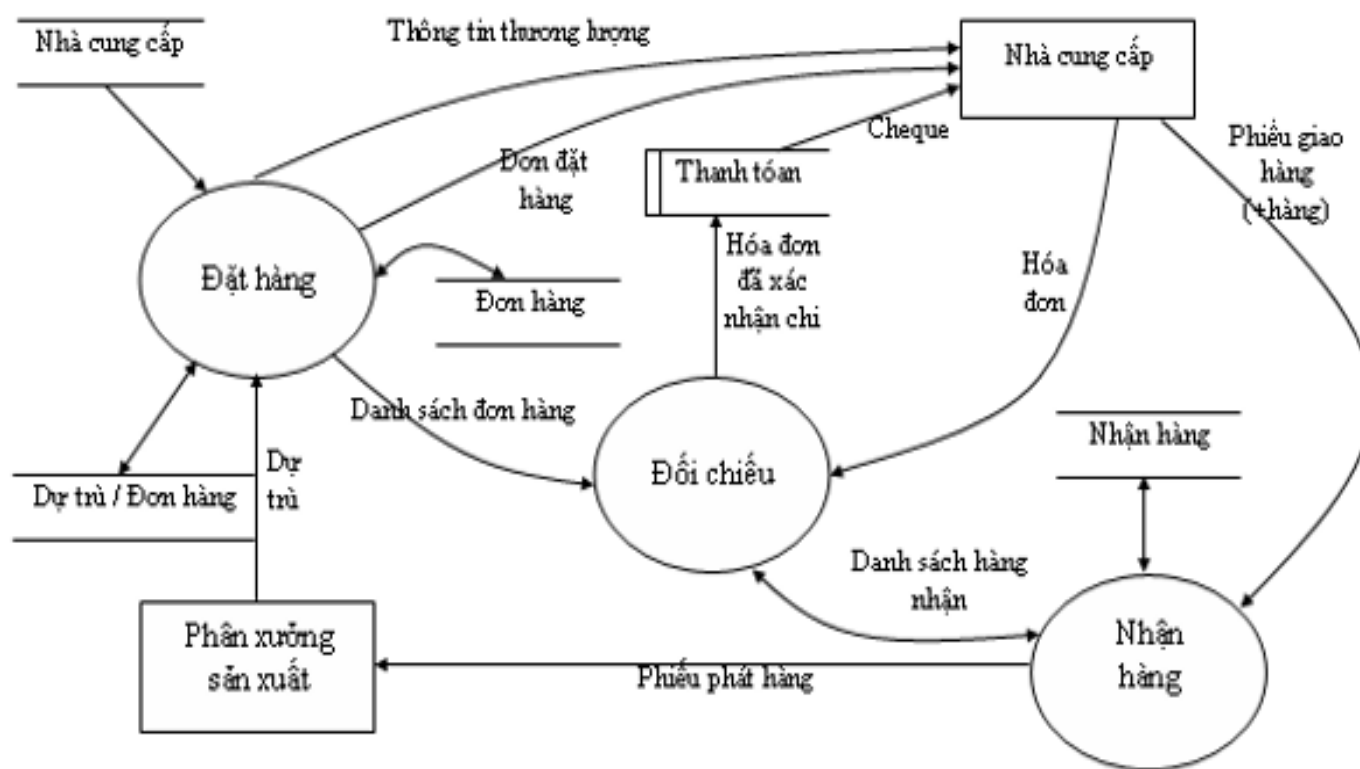
3. Các phương pháp thiết kế hệ thống theo cấu trúc hóa (tiếp)

- Minh họa dưới đây sẽ giải thích cho việc phân chia chức năng theo bong bóng của DFD (biểu đồ luồng dữ liệu)



Biểu đồ luồng dữ liệu

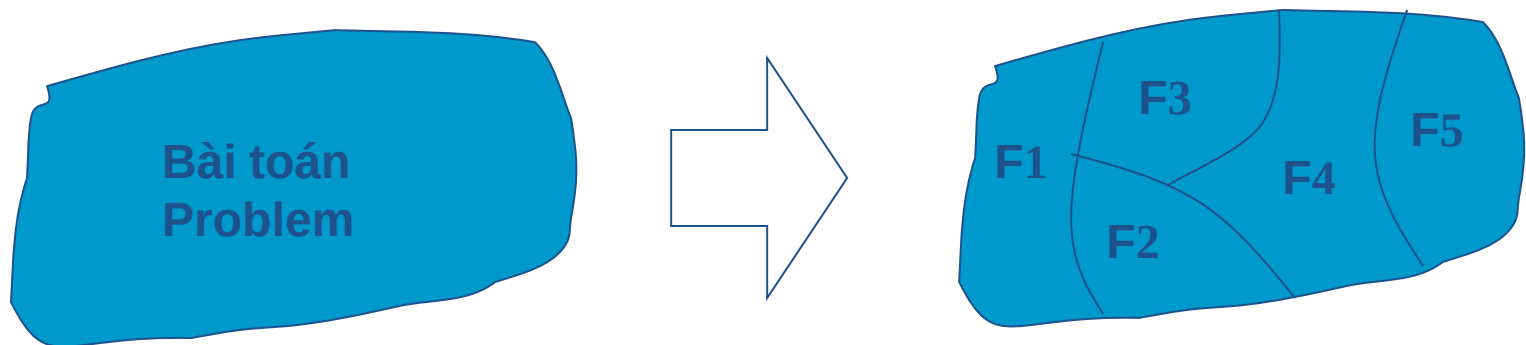
- Ví dụ biểu đồ luồng dữ liệu của hệ cung ứng vật tư





3. Các phương pháp thiết kế hệ thống theo cấu trúc hóa

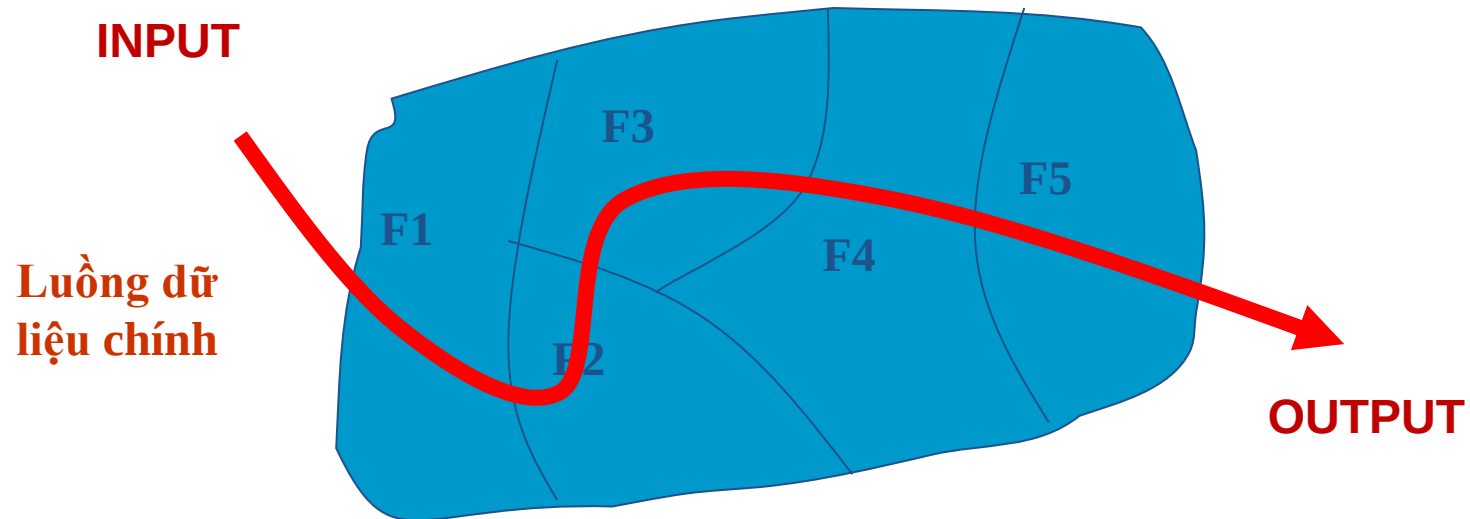
- Phương pháp dựa vào trung tâm biến đổi STS: Chia thành 8 bước
- B1: Chia đối tượng “bài toán” thành các chức năng thành phần





Phương pháp dựa vào trung tâm biến đổi STS

B2: Tìm ra luồng dữ liệu chính đi qua các chức năng: từ đầu vào (Input) tới đầu ra (Output) – Quyết định luồng dữ liệu





Phương pháp dựa vào trung tâm biến đổi STS

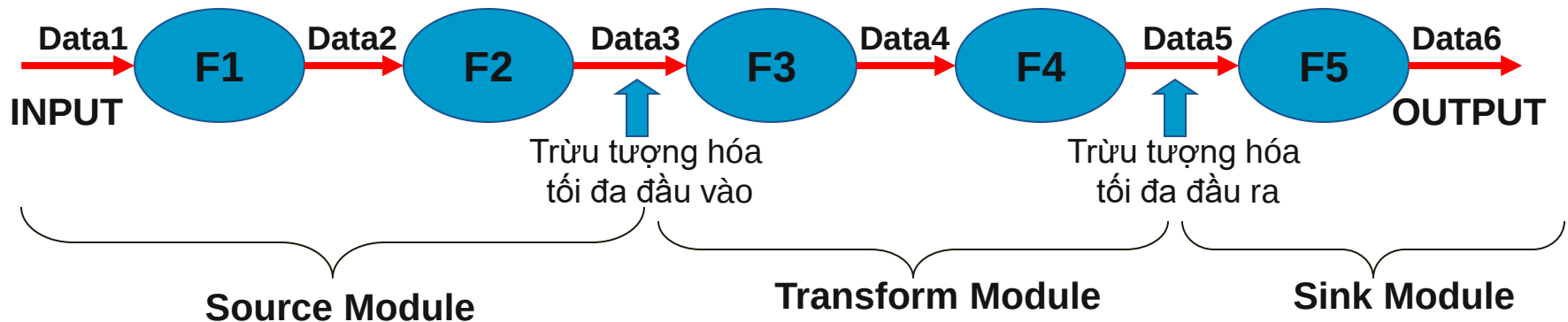
B3: Theo luồng dữ liệu chính: thay từng chức năng bởi bong bóng và làm rõ dữ liệu giữa các bong bóng– Quyết định bong bóng và dữ liệu





Phương pháp dựa vào trung tâm biến đổi STS

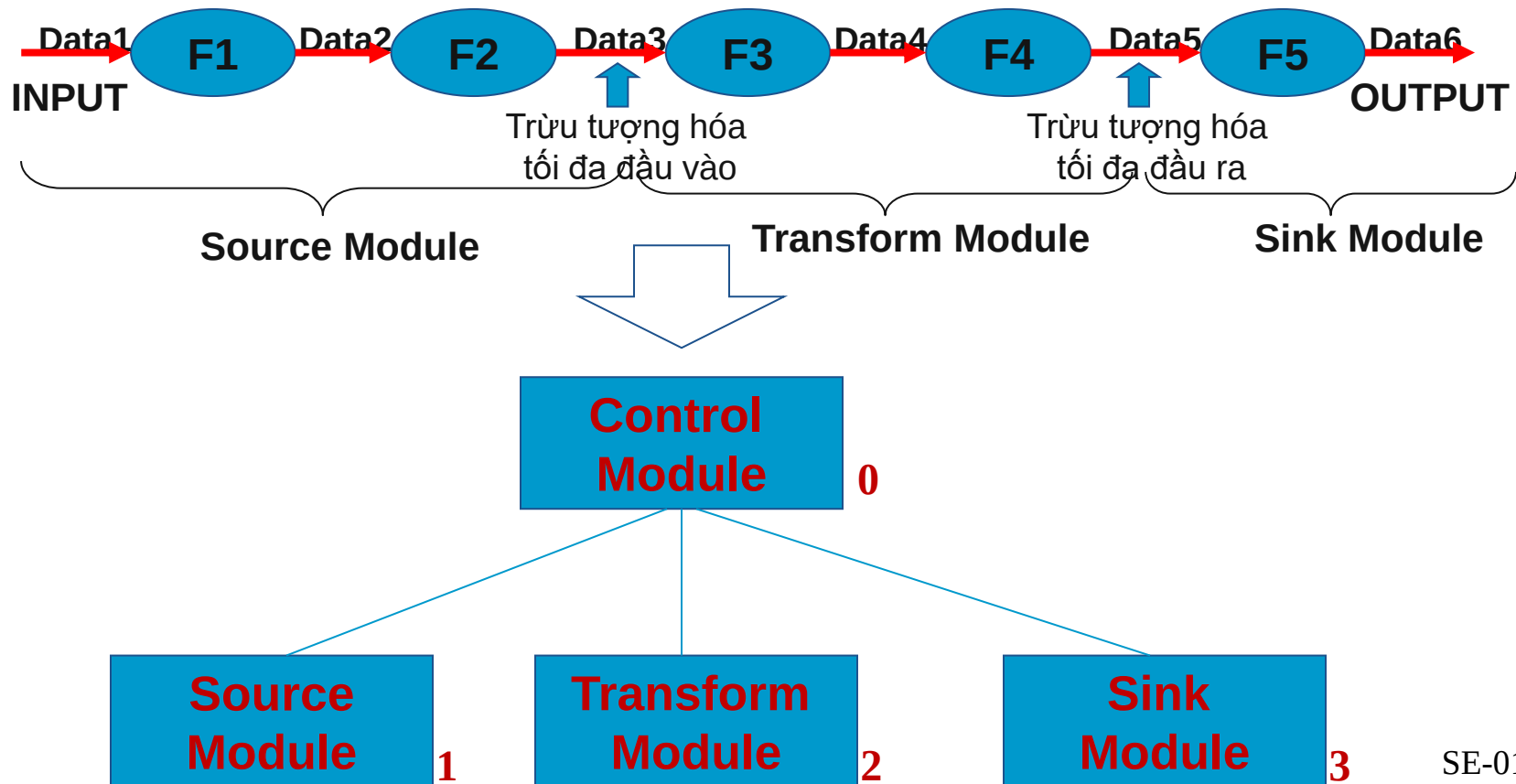
B4: Xác định vị trí trừu tượng hóa tối đa đầu vào và đầu ra





Phương pháp dựa vào trung tâm biến đổi STS

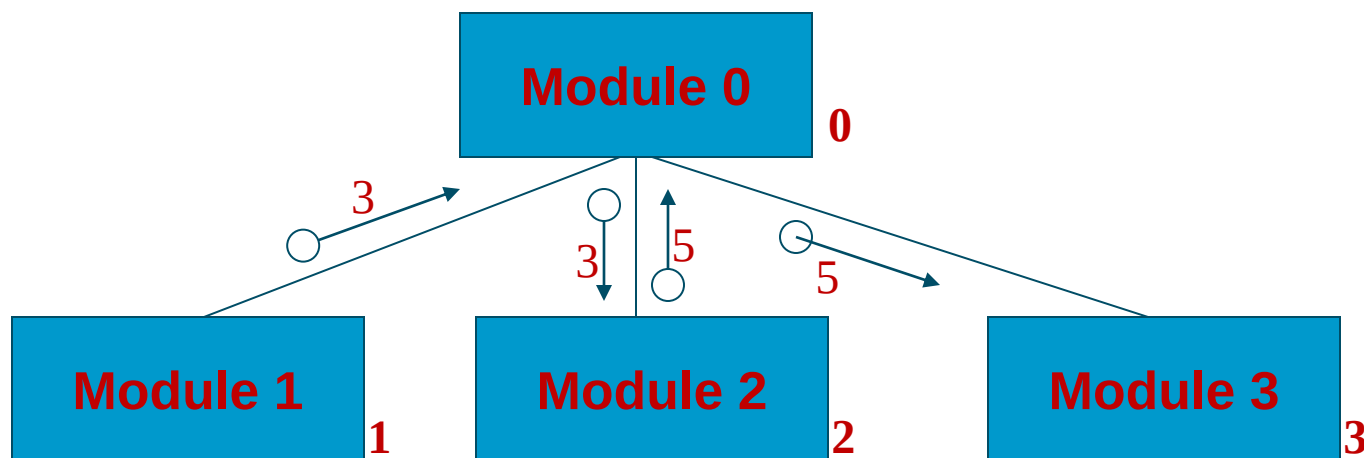
B5: Chuyển sang sơ đồ phân cấp





Phương pháp dựa vào trung tâm biến đổi STS

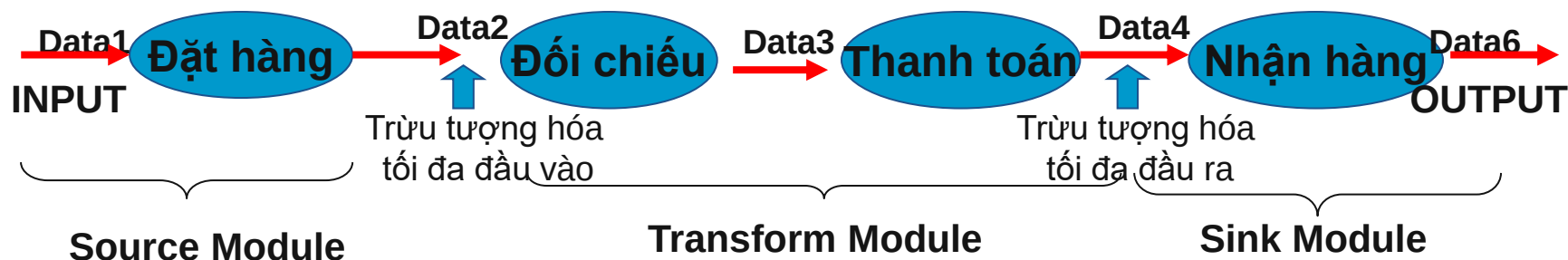
B6: Xác định các tham số giữa các mô đun dựa theo quan hệ phụ thuộc





Phương pháp dựa vào trung tâm biến đổi STS

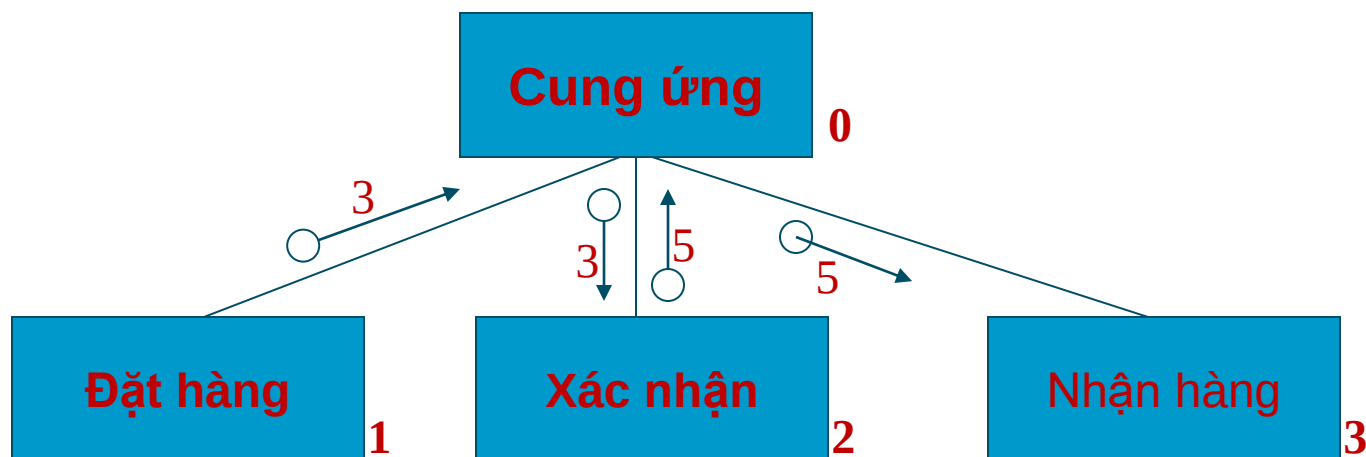
- Hệ cung ứng vật tư





Phương pháp dựa vào trung tâm biến đổi STS

- Hệ cung ứng vật tư





Phương pháp dựa vào trung tâm biến đổi STS

B7: Với từng môđun (Source, Transform, Sink) lại áp dụng cách phân chia STS lặp lại các bước từ 1) đến 6). Đôi khi có trường hợp không chia thành 3 môđun nhỏ mà thành 2 hoặc 1

B8: Tiếp tục chia đến mức cấu trúc logic khi môđun tương ứng với thuật toán đã biết thì dừng. Tổng hợp lại ta được cấu trúc phân cấp: mỗi nút là 1 môđun với số nhánh phía dưới không nhiều hơn 3



Phương pháp phân chia giao dịch TR

- Khi không tồn tại luồng dữ liệu chính, mà dữ liệu vào có đặc thù khác nhau như những nguồn khác nhau xem như các *Giao dịch* thì ta áp dụng phương pháp phân chia giao dịch TR.

Nguyên tắc:

- Mỗi giao dịch ứng với 1 môđun xử lý nó
- Phân chia môđun có thể: theo kinh nghiệm; theo tính độc lập môđun; theo số bước tối đa trong 1 môđun (ví dụ < 50) và theo chuẩn.



Phương pháp phân chia giao dịch TR

Các bước thực hiện

B1: Xác định trung tâm giao dịch gồm một số chức năng phân loại và các chức năng tham gia vào quá trình xử lý:

- Vào: một số chức năng truyền dẫn thông tin từ nguồn và một số chức năng sơ chế.
- Ra: Một số chức năng dẫn thông tin ra từ các tình huống xử lý.

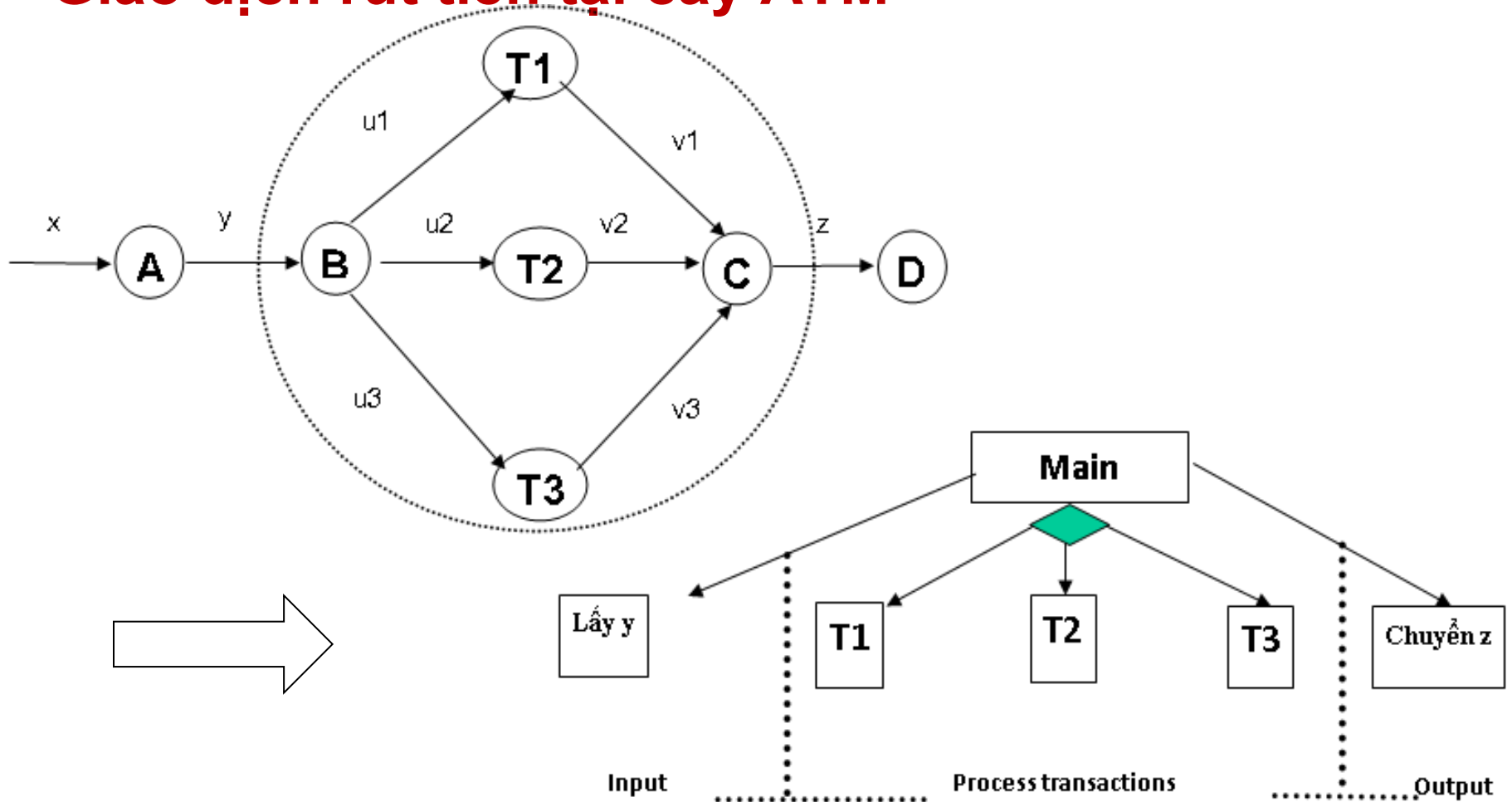
B2: Vẽ 2 mức cao nhất của lưu đồ cấu trúc:

- Một mô đun mức cao nhất (đỉnh): mức 1
- 1 mô đun vào, mỗi mô đun xử lý cho mỗi trường hợp và 1 mô đun ra: mức 2.



Phương pháp phân chia giao dịch TR

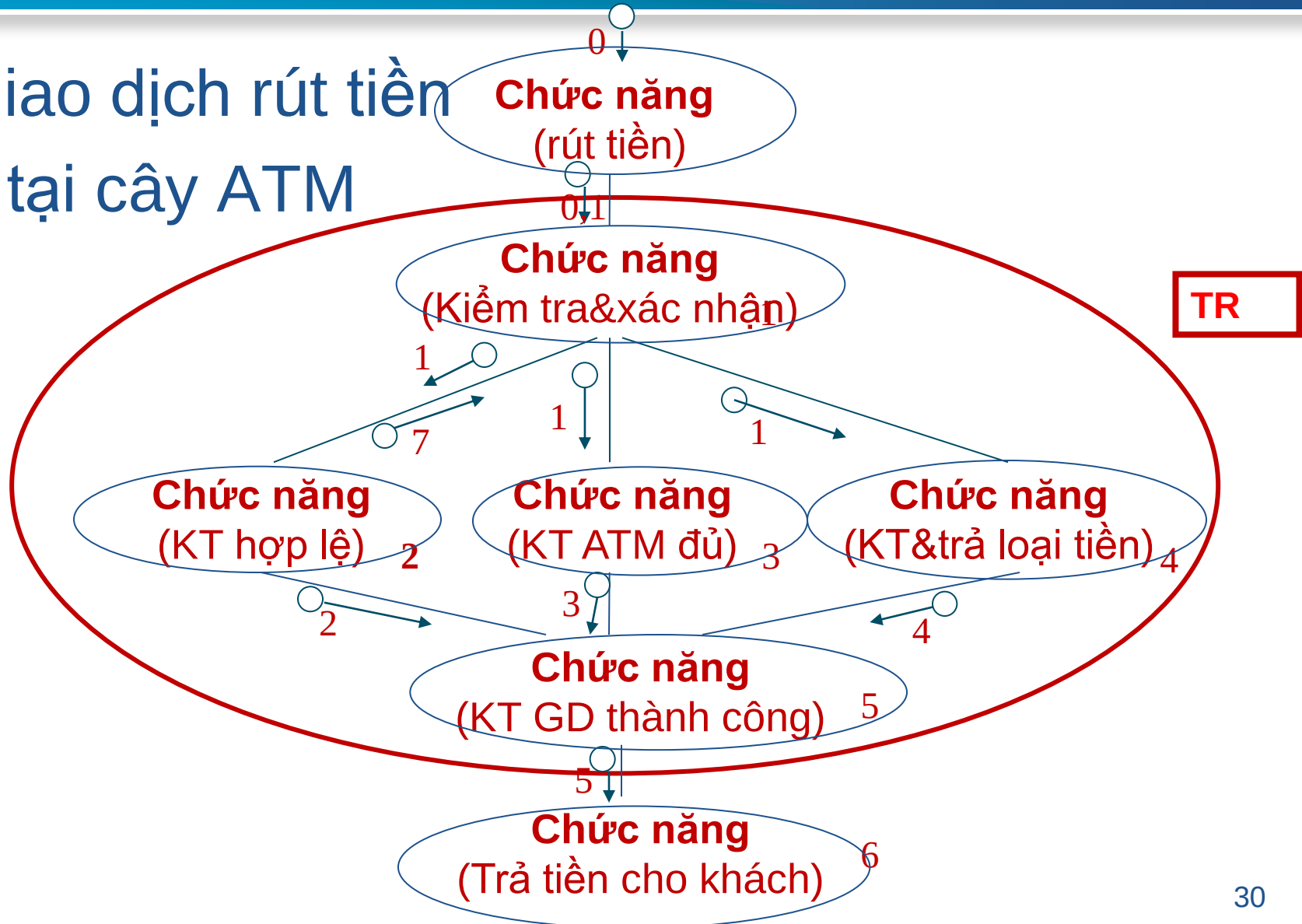
Giao dịch rút tiền tại cây ATM





Thiết kế hệ thống hướng cấu trúc

- Giao dịch rút tiền tại cây ATM





Đặc trưng của thiết kế cấu trúc hóa

- Dễ thích ứng với mô hình vòng đời thác nước do tính thân thiện cao
- Thiết kế theo tiến trình, không hợp với thiết kế xử lý theo lô (batch system)
- Dùng phân chia - kết hợp để giải quyết tính phức tạp của hệ thống
- Top-down trong phân chia môđun
- Kỹ thuật lập trình hiệu quả



Các điểm cần lưu ý khi thiết kế cấu trúc hóa

- (1) Có thể trích được luồng dữ liệu từ hệ thống: đó là phần nội dung đặc tả yêu cầu và giao diện.
- (2) Xem xét tối ưu tài nguyên kiến trúc lên hệ thống rồi quyết định kiến trúc.
- (3) Theo quá trình biến đổi dữ liệu, hãy xem những chức năng được kiến trúc như thế nào.



Các điểm cần lưu ý khi thiết kế cấu trúc hóa

- (4) Từ kiến trúc các chức năng theo (3), hãy xem xét và chỉnh lại, từ đó chuyển sang kiến trúc chương trình và thiết kế chi tiết
- (5) Quyết định các đơn vị chương trình theo các chức năng của hệ phần mềm có dựa theo luồng dữ liệu và phân chia ra các thành phần
- (6) Khi cấu trúc chương trình lớn quá, phải phân chia nhỏ hơn thành các môđun



Các điểm cần lưu ý khi thiết kế cấu trúc hóa

- (7) Xem xét dữ liệu vào-ra và các tệp dùng chung của chương trình. Truy cập tệp tối ưu.
- (8) Hãy nghĩ xem để có được những thiết kế trên thì nên dùng phương pháp luận và những kỹ thuật gì ?



II. Phương pháp hướng đối tượng

- Cách tiếp cận hướng đối tượng
- Đặc trưng và các ưu nhược điểm
- Các giai đoạn của phương pháp hướng đối tượng:
 - Phân tích hướng đối tượng
 - Thiết kế hướng đối tượng
 - Cài đặt hướng đối tượng (sẽ được học trong môn lập trình hướng đối tượng)
 - Kiểm thử hướng đối tượng (sẽ được học trong chương cuối)



Thiết kế hướng đối tượng

- Là lối tiếp cận mà tư duy vấn đề theo lối ánh xạ các thành phần bài toán vào các đối tượng ngoài đời thực. Chúng ta chia ứng dụng thành các thành phần nhỏ gọi là các đối tượng, chúng tương đối độc lập với nhau.
- Mỗi đối tượng là các kết hợp giữa code và dữ liệu mà chúng được nhìn nhận như là một đơn vị duy nhất, nó có tên riêng và tham chiếu đến nó qua tên. Nó có khả năng nhận vào các thông báo, xử lý dữ liệu (bên trong của nó), và gửi ra hay trả lời đến các đối tượng khác hay đến môi trường.



Đặc trưng của thiết kế hướng đối tượng

- Không nên có vùng dữ liệu dùng chung. Các đối tượng liên lạc với nhau bằng cách trao đổi thông báo chứ không phải các biến dùng chung.
- Các đối tượng là các thực thể độc lập, dễ thay đổi vì rằng tất cả các trạng thái các thông tin biểu diễn chỉ ảnh hưởng trong phạm vi chính đối tượng đó thôi. Các thay đổi về biểu diễn thông tin có thể được thực hiện không cần có sự tham khảo tới các đối tượng hệ thống nào khác.
- Các đối tượng có thể phân tán và có thể hành động tuần tự hoặc song song.



Ưu điểm của thiết kế hướng đối tượng

- Dễ bảo trì, thay đổi trong khi thực hiện một đối tượng hoặc thêm các dịch vụ sẽ không làm thay đổi hay ảnh hưởng tới các đối tượng hệ thống khác.
- Các đối tượng là các thành phần dùng lại được. Một thiết kế có thể dùng lại các đối tượng đã được thiết kế trong các bản thiết kế trước đó.
- Có các lớp hệ thống thể hiện quan hệ rõ ràng giữa các thực thể có thực (như các thành phần phần cứng) với các đối tượng điều khiển nó trong hệ thống. Điều này đạt được tính dễ hiểu trong thiết kế.



Nhược điểm của thiết kế hướng đối tượng

- Việc minh định các đối tượng trong hệ thống là khó khăn. Cách nhìn tự nhiên là cách nhìn hướng chức năng và việc thích nghi với cách nhìn hướng đối tượng đôi khi là khó khăn.
- Thiếu tính khả mở và tiến hóa.



Các giai đoạn của phương pháp hướng đối tượng

- Giai đoạn phân tích: Mô tả về vấn đề của dự án, xem xét và đưa ra giải pháp. Kết thúc giai đoạn này người dùng và nhà thiết kế thống nhất với nhau về chức năng của hệ thống, nhà phát triển sẽ cố gắng xác định các yêu cầu quan trọng.
- Giai đoạn thiết kế: Tạo ra bản thiết kế(blueprint) của hệ thống. Bước đầu tiên là xác định các lớp và mối liên hệ giữa chúng.
- Giai đoạn thực hiện: Chọn một ngôn ngữ lập trình hướng đối tượng để thực hiện.
- Giai đoạn kiểm thử: Kết hợp các loại kiểm thử hướng đối tượng



Phân tích hệ thống theo hướng đối tượng

- Là kỹ thuật đặc tả bán hình thức trong các kỹ thuật hướng đối tượng. Có 3 bước chính:
 - Mô hình các trường hợp sử dụng
 - Mô hình lớp
 - Mô hình động



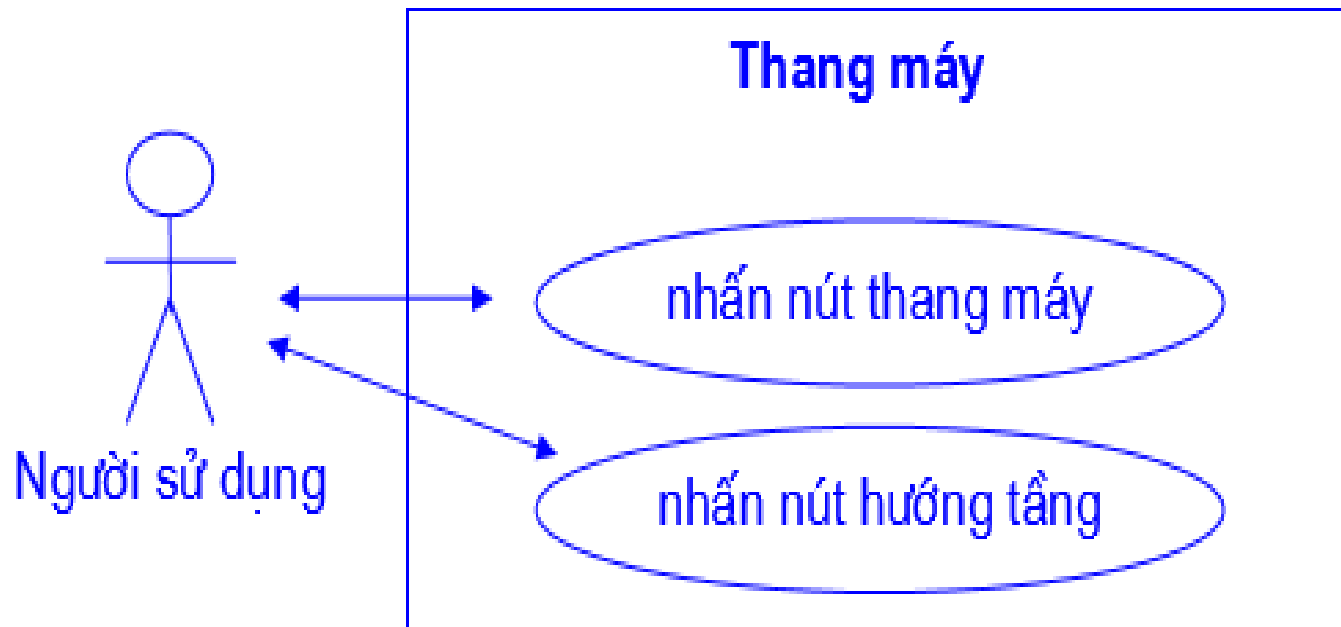
Phân tích hệ thống theo hướng đối tượng

- Mô hình các trường hợp sử dụng
 - Xác định các chức năng của sản phẩm, cung cấp cái nhìn trên tổng thể các chức năng nên còn gọi là mô hình chức năng.
 - Thể hiện sự tương tác giữa các lớp trong sản phẩm với người sử dụng.
 - Trình bày dưới hai dạng: Sơ đồ các trường hợp sử dụng và các kịch bản kết hợp.
 - Kịch bản là thể hiện của trường hợp sử dụng.



Mô hình các trường hợp sử dụng

- Ví dụ: Các trường hợp sử dụng trong bài toán thang máy





Mô hình các trường hợp sử dụng

1. Người A nhấn nút UP tại tầng 3 để yêu cầu thang máy. Người A muốn đi đến tầng 7.
2. Nút UP sáng lên.
3. Một thang máy đến tầng 3. Trong thang máy này có người B vào thang máy từ tầng 1 và đi đến tầng 9.
4. Nút UP tắt.
5. Thang máy mở cửa, người A vào thang máy.
6. Người A nhấn nút đi tầng 7.
7. Nút tầng 7 sáng lên.
8. Cửa thang máy đóng.
9. Thang máy đi đến tầng 7.
10. Nút tầng 7 tắt.
11. Cửa thang máy mở cho phép người A đi ra khỏi thang máy.
12. Bộ định thời hoạt động. người A đi ra ngoài.
13. Cửa thang máy đóng lại sau khi đã hết thời gian.
14. Thang máy tiếp tục đi đến tầng 9 với người B.

Một kịch bản thông thường



Mô hình các trường hợp sử dụng

1. Người A nhấn nút UP tại tầng 3 để yêu cầu thang máy. Người A muốn đi đến tầng 1.
2. Nút UP sáng lên.
3. Một thang máy đến tầng 3. Trong thang máy này có người B vào thang máy từ tầng 1 và đi đến tầng 9.
4. Nút UP tắt.
5. Thang máy mở cửa, người A vào thang máy.
6. Người A nhấn nút đi tầng 1.
7. Nút tầng 1 sáng lên.
8. Cửa thang máy đóng.
9. Thang máy đi đến tầng 9.
10. Nút tầng 9 tắt.
11. Cửa thang máy mở cho phép người B đi ra khỏi thang máy.
12. Bộ định thời hoạt động. người B đi ra ngoài.
13. Cửa thang máy đóng lại sau khi đã hết thời gian.
14. Thang máy tiếp tục đi đến tầng 1 với người A.

Một kịch bản không thông thường



Mô hình lớp

- Tách ra các lớp, các thuộc tính và các thể hiện bằng sơ đồ thực thể quan hệ.
- Tách tên(noun extraction) nhằm chỉ ra các lớp có thể có với 3 bước sau:
 - Định nghĩa súc tích vấn đề.
 - Chiến lược không hình thức: Lấy ra các ràng buộc một cách không hình thức để giải quyết vấn đề
 - Hình thức hóa chiến lược: Xác định các tên trong chiến lược không hình thức(kể cả các nội dung nằm ngoài biên vấn đề) và sử dụng các tên như là các ứng cử viên cho tên của các lớp.



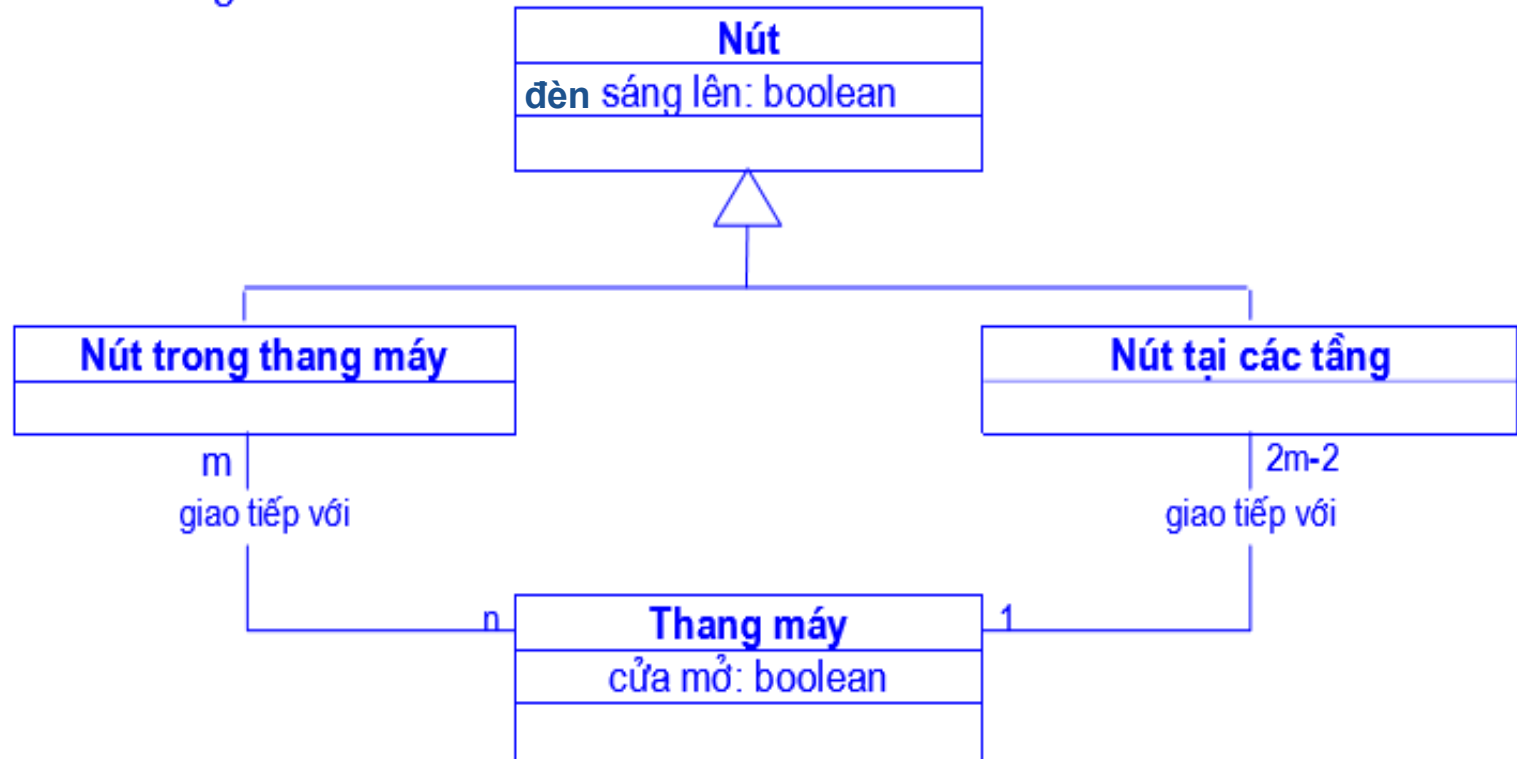
Mô hình lớp

- Khung lớp trách nhiệm và hợp tác CRC (class-responsibility-collaboration) do nhóm phát triển điền vào các thông tin:
 - Tên lớp
 - Các chức năng (trách nhiệm)
 - Danh sách các lớp gọi các chức năng này (hợp tác)



Mô hình lớp

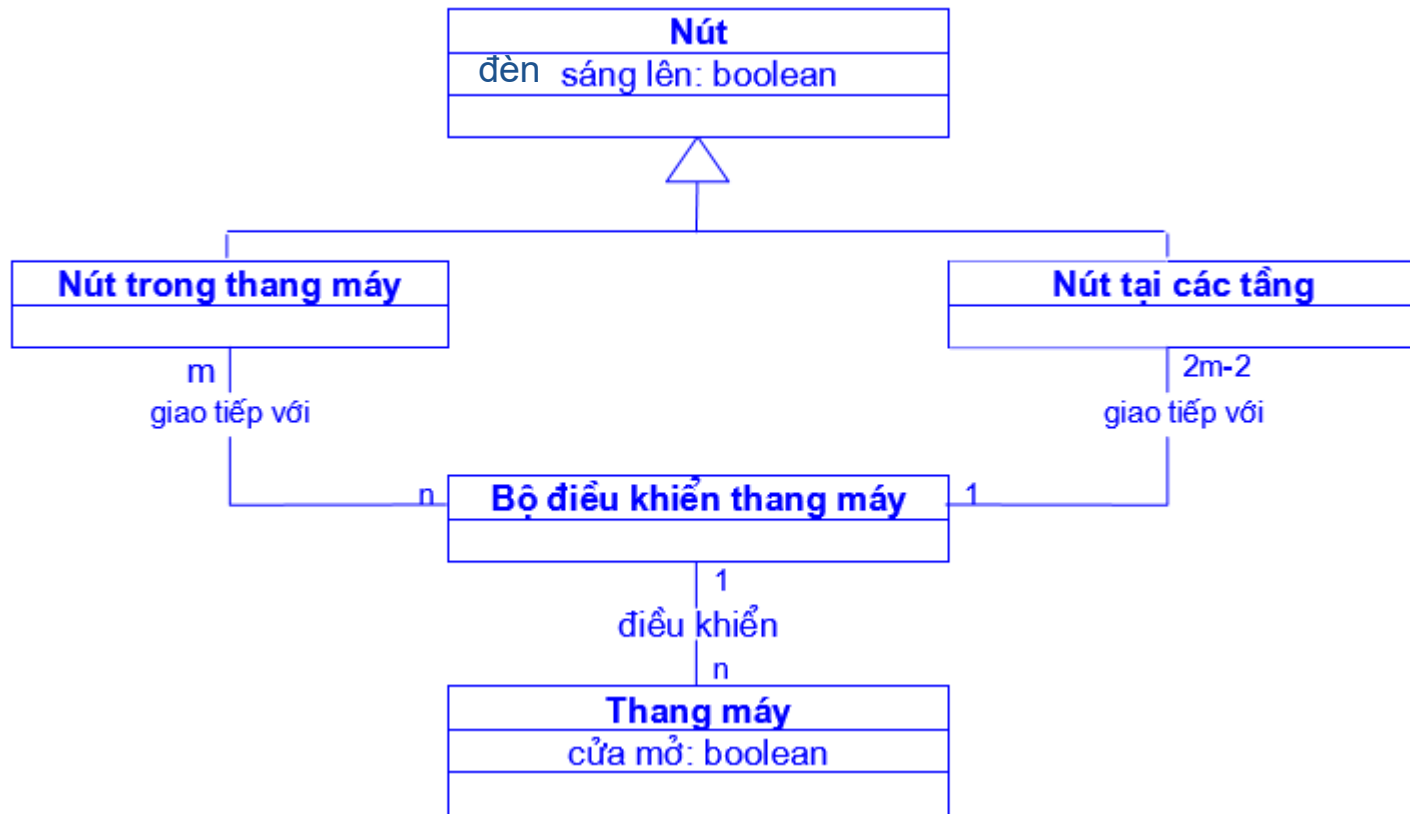
- Ví dụ: Bài toán thang máy



Sơ đồ lớp đầu tiên



Mô hình lớp



Sơ đồ lớp thứ hai



Mô hình lớp

LỚP Bộ điều khiển thang máy
TRÁCH NHIỆM 1. Bật nút trong thang máy 2. Tắt nút trong thang máy 3. Bật nút tại tầng 4. Tắt nút tại tầng 5. Mở cửa thang máy 6. Đóng cửa thang máy 7. Di chuyển thang máy lên một tầng 8. Di chuyển thang máy xuống một tầng
HỢP TÁC 1. Lớp Nút trong thang máy 2. Lớp Nút tại các tầng 3. Lớp Thang máy

CRC đầu tiên của lớp *bộ điều khiển thang máy*



Mô hình lớp

- Ưu điểm:
 - Được thực hiện bởi một nhóm làm việc do đó giảm thiểu các thiếu sót hay các mục không chính xác trong lớp.
 - Dễ dàng xác định sơ đồ lớp có đầy đủ và chính xác hay chưa.
 - Rất tốt trong việc xác định các chức năng và khả năng hợp tác giữa các lớp.
- Nhược điểm: Không phải là cách tìm lớp tốt nếu như nhóm làm việc chưa có kinh nghiệm trên các lĩnh vực liên quan.



Mô hình động

- Xây dựng sơ đồ trạng thái là việc mô tả sản phẩm dưới dạng mô hình trạng thái hữu hạn: Trạng thái, sự kiện, vị từ.
- Thực hiện trên từng lớp
- Tương tự FSM



Mô hình động



Sơ đồ trạng thái cho lớp *bộ điều khiển thang máy*

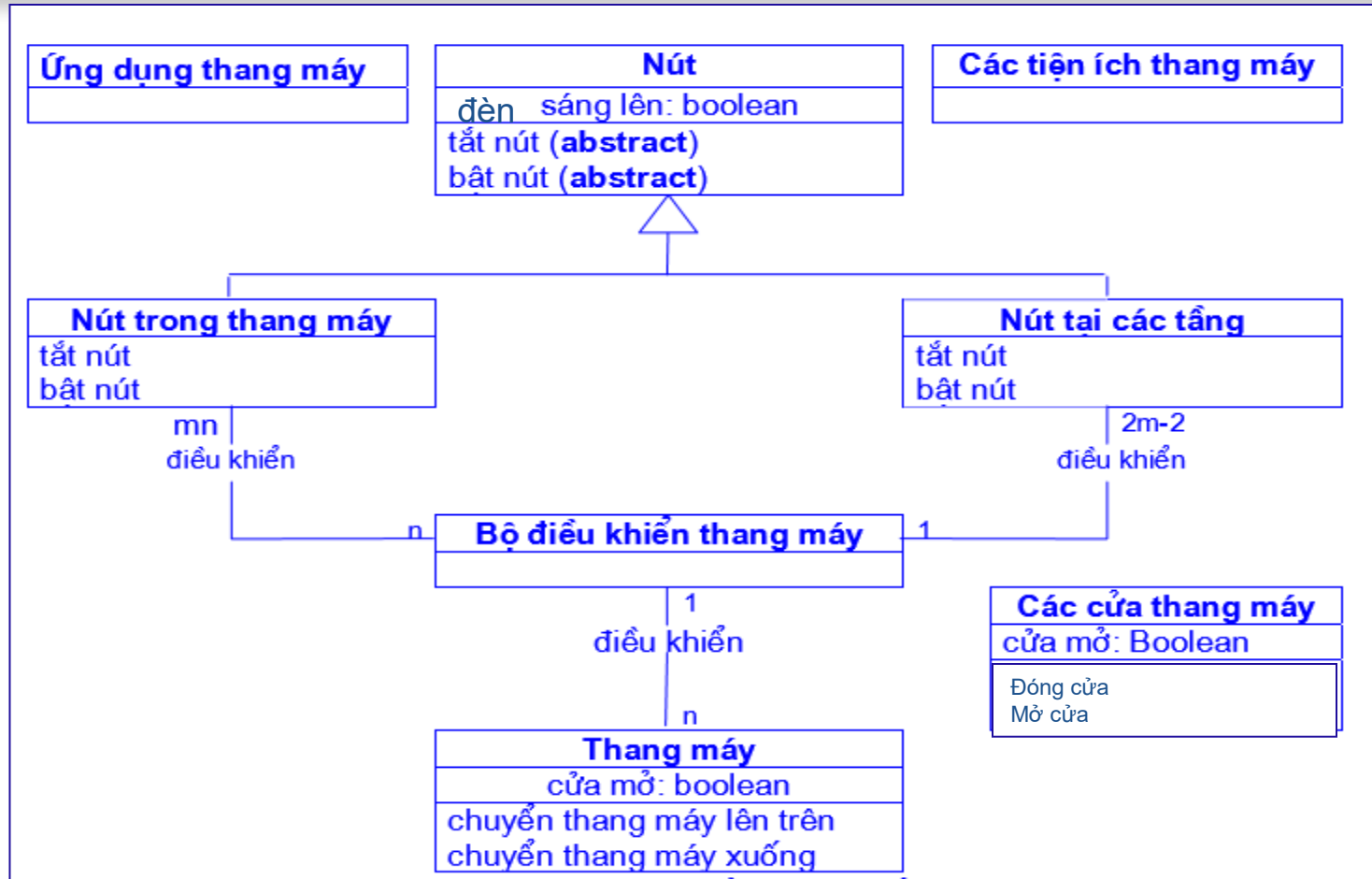


Thiết kế hệ thống hướng đối tượng

- Thiết kế sản phẩm thành các đối tượng (object) là các thể hiện của các lớp(class) hay các lớp con (subclass).
- Bao gồm bốn bước:
 - Xây dựng sơ đồ tương tác cho từng kịch bản
 - Xây dựng sơ đồ lớp chi tiết
 - Thiết kế sản phẩm theo các đối tượng của khách hàng
 - Tiến hành thiết kế chi tiết



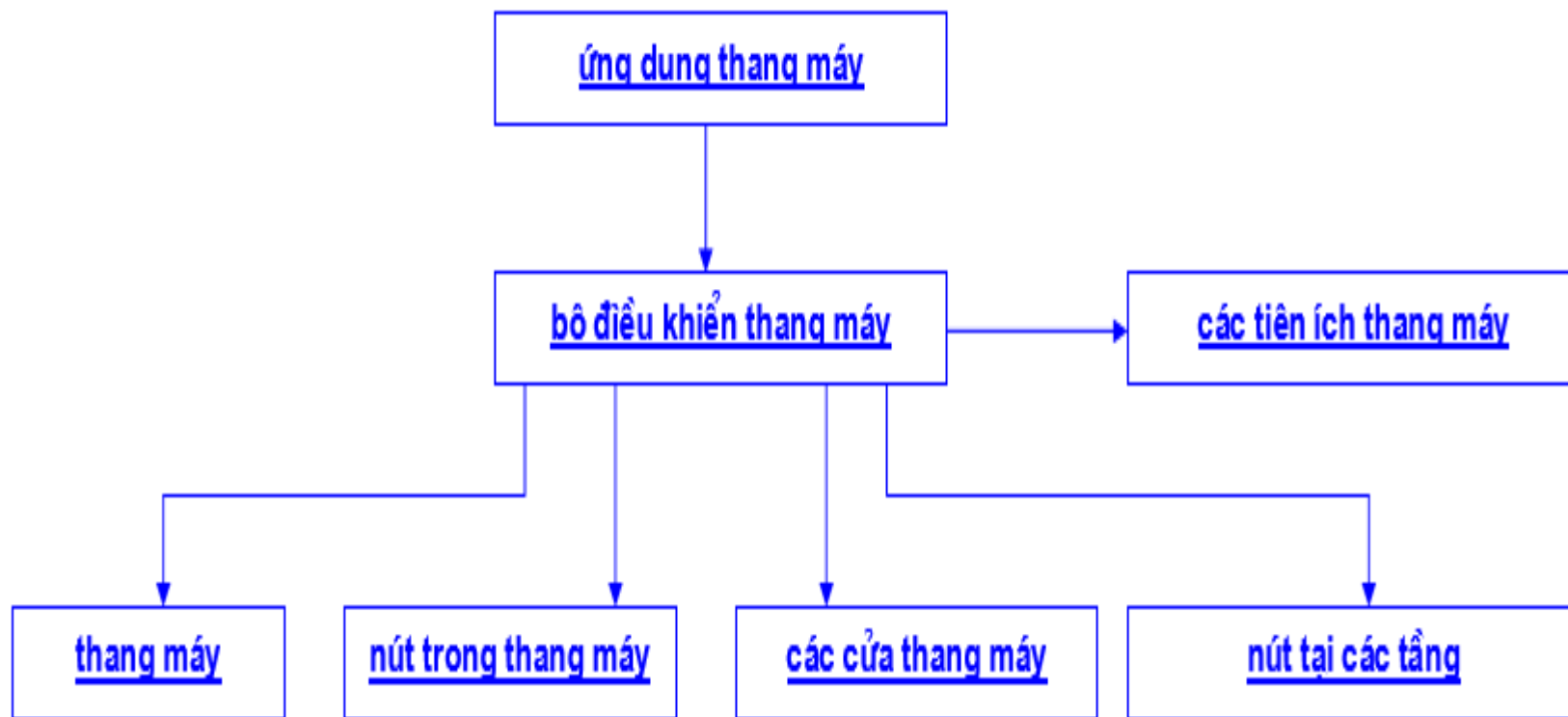
Thiết kế hệ thống hướng đối tượng



Sơ đồ lớp chi tiết



Thiết kế hệ thống hướng đối tượng



Quan hệ khách hàng-đối tượng



III. Phương pháp hướng tương quan

- Phương pháp luận của phương pháp hướng tương quan.
- Ưu điểm và nhược điểm.
- Quản lý các concern của kỹ nghệ yêu cầu hệ thống.
 - Định nghĩa các Concern.
 - Quản lý các concern của hệ thống.
- Thiết kế hệ thống hướng tương quan (Aspect-Oriented design).



Phương pháp luận

- Phương pháp hướng tương quan là phương pháp sử dụng mô hình AOP (Aspect-Oriented Programming).
- Các nhiệm vụ đan xen (*cross-cutting concerns*) được hệ thống đóng gói trong các aspect.
- AOP sẽ quyết định các module khác nhau (gồm các core concern và các aspect) sẽ được gắn kết với nhau như thế nào để tạo nên hệ thống cuối cùng.



Ưu và nhược điểm

- Đây là cách suy nghĩ về thiết kế và cài đặt theo một phương pháp mới với các ưu điểm sau:
 - Tách biệt hơn các chức năng của các module độc lập
 - Tính module hóa cao hơn
 - Phát triển hệ thống dễ dàng hơn
 - Kết nối muện thiết kế
 - Tính tái sử dụng mã
 - Giảm thời gian cài đặt và giá thành phần mềm



Ưu và nhược điểm

- Một số đặc điểm gây trở ngại:
 - Chưa giải quyết được vấn đề của phương pháp OOP về tính khả mở và tiến hóa. Nó chỉ đưa ra cách giải quyết bài toán tốt hơn theo hai hướng phương pháp cấu trúc và OOP.
 - Không giải quyết được cho các thiết kế cầu thả.
 - Phá vỡ tính đóng gói trong OOP (encapsulation)



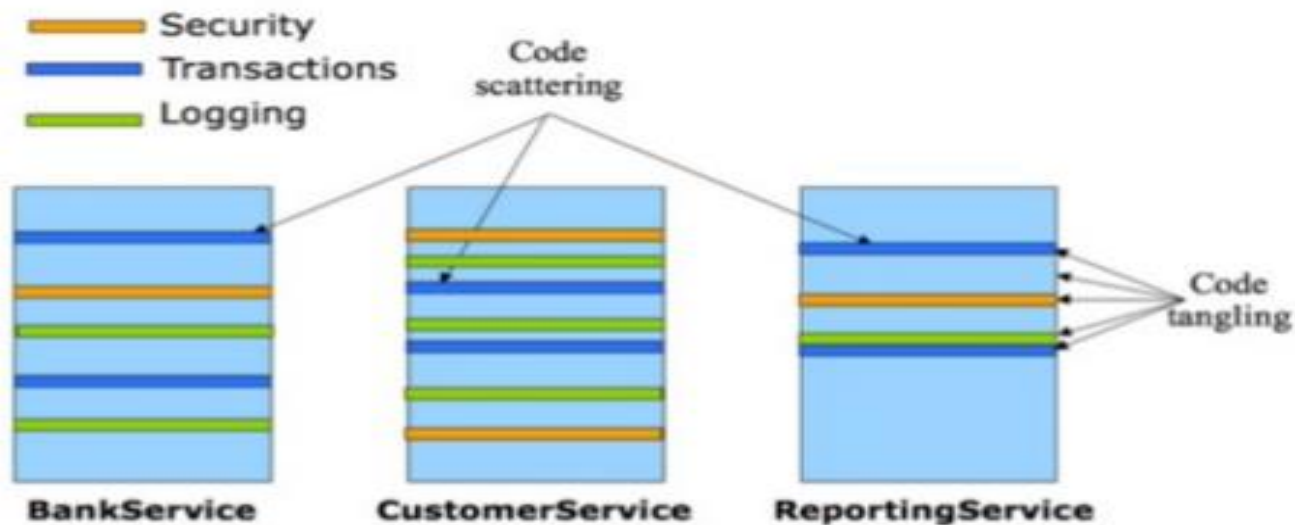
Định nghĩa về concern

- Concern là các yêu cầu cụ thể hay mối quan tâm đặc trưng được xác định để thoả mãn mục tiêu chung của hệ thống.
- Hệ thống phần mềm là sự gắn kết của tập các concern. Mỗi yêu cầu hệ thống sẽ được phân tách thành nhiều nhiệm vụ, mỗi nhiệm vụ đó sẽ là một concern:
 - Các core concern thường là các logic nghiệp vụ (business logic).
 - aspect sẽ gồm các cross-cutting concern tương tác với các core concern.



Quản lý các concern

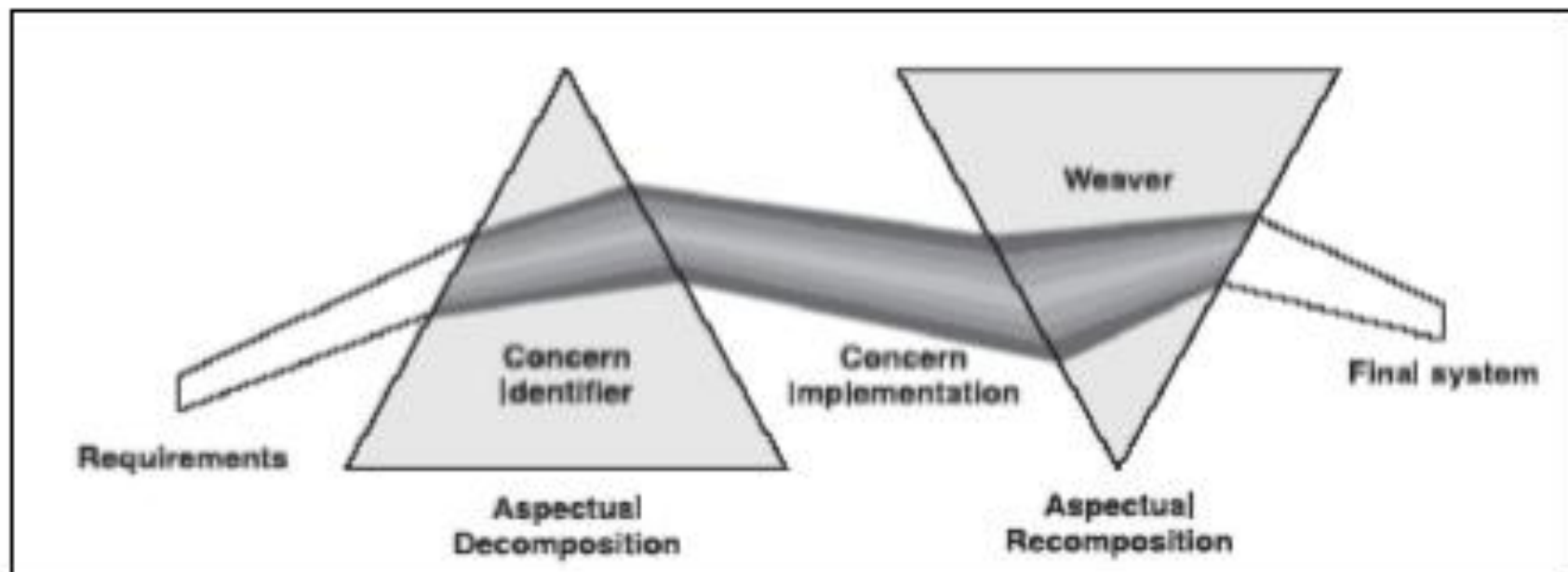
- Giải quyết hai vấn đề của việc tương tác giữa các nhiệm vụ bằng việc phân tách các concern trong aspect:
 - Tán xạ(Scattering): Phân mảnh code
 - Lộn xộn(Tangle): Rối trộn code





Quản lý các concern

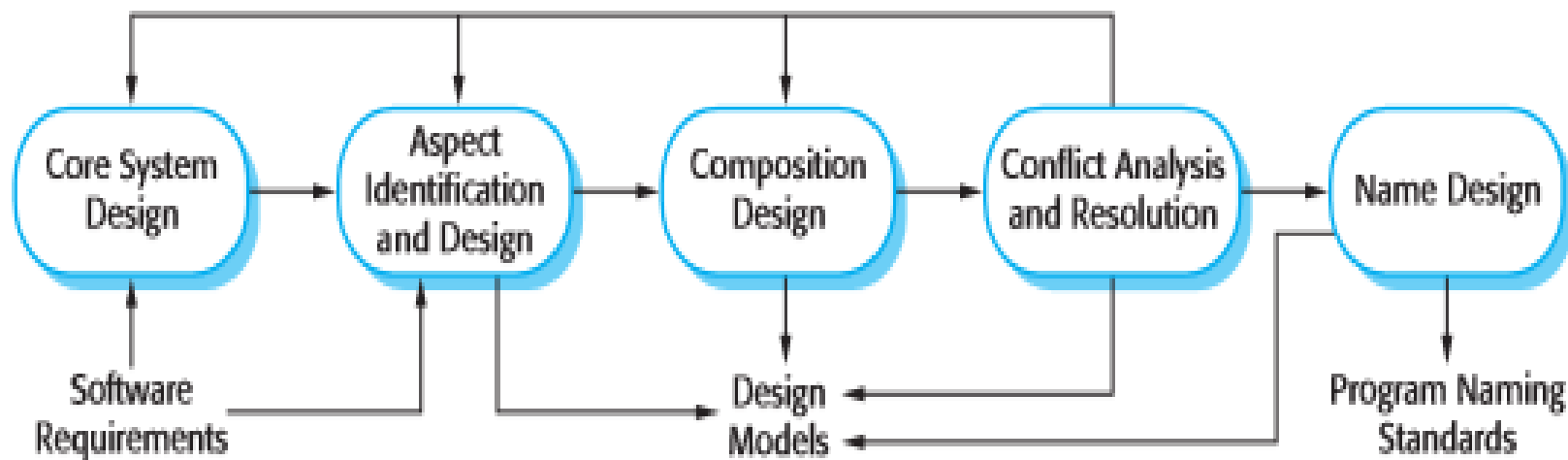
- Các giai đoạn phát triển sử dụng phương pháp hướng tương quan





Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

- Quá trình thiết kế hệ thống hướng tương quan





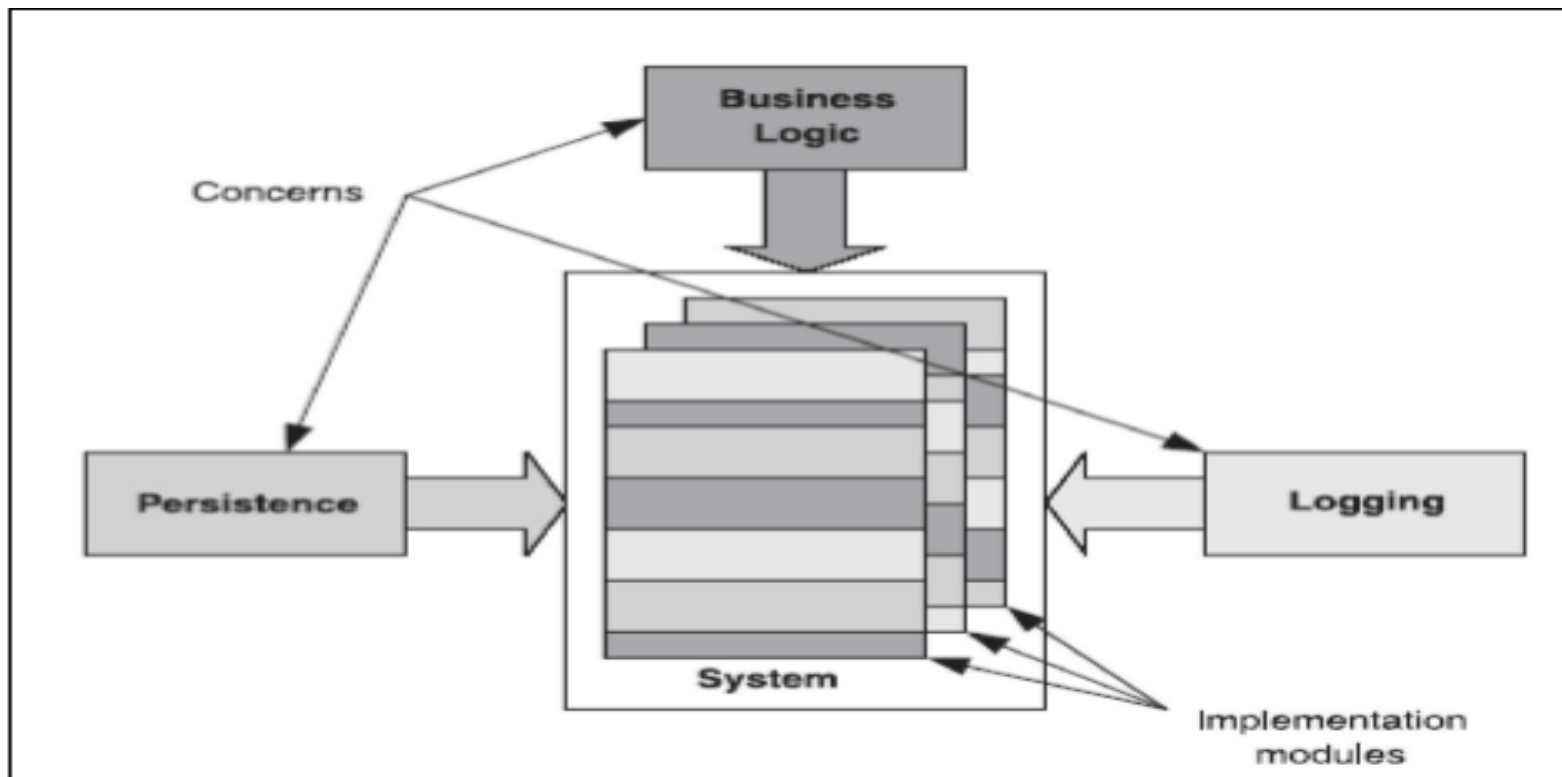
Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

- Một số bước điển hình sử dụng AOP trong pha thiết kế:
 - B1: Nhận biết các concern đan xen: Bước này là một phần trong việc ánh xạ các yêu cầu người dùng tới các module.
 - B2: Thiết kế các concern lõi trước: Áp dụng các quy tắc và phương pháp truyền thống để thiết kế các concern lõi.
 - B3: Thiết kế các concern đan xen: Xác định các concern đan xen cần thiết, dễ thấy.



Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

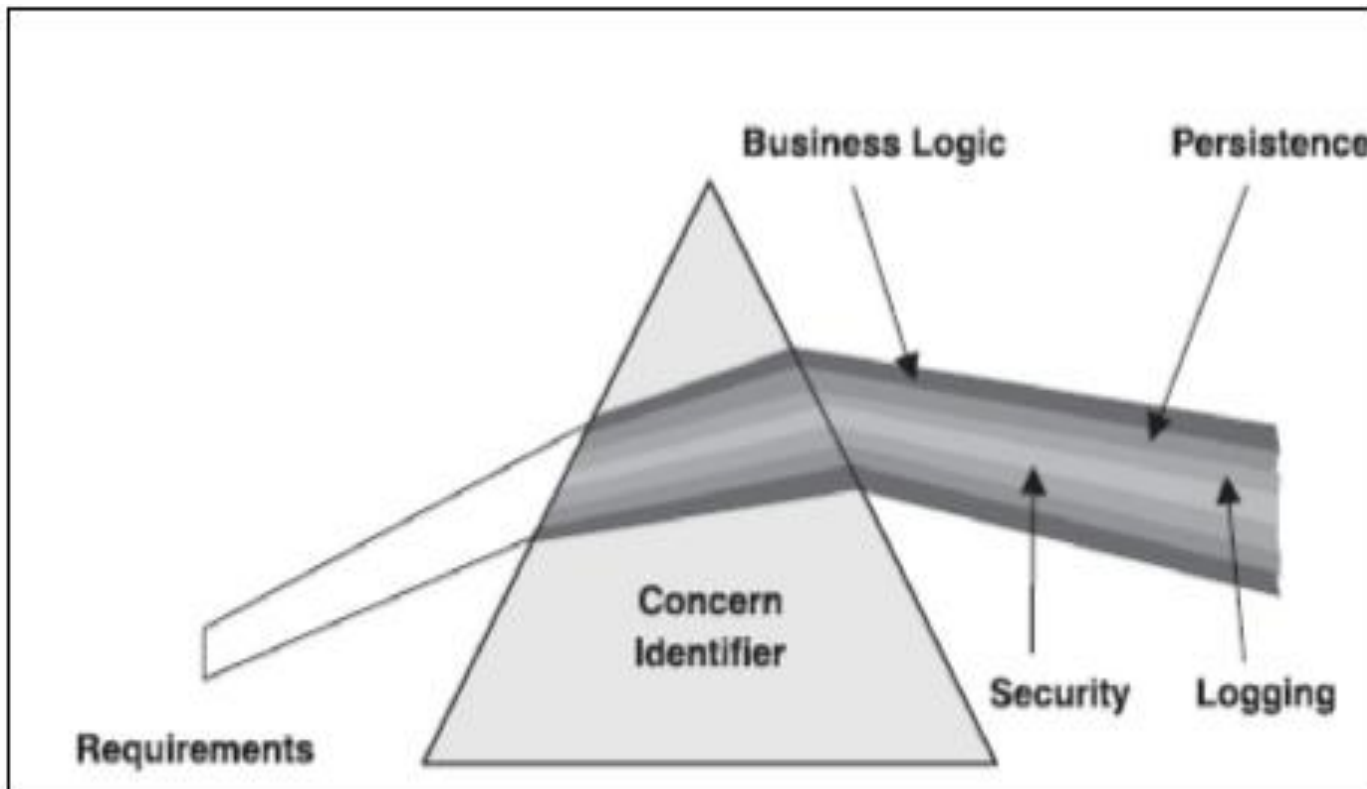
- Ví dụ xây dựng mô hình tương quan với nhiều concern, sau đó ánh xạ vào code hướng đối tượng truyền thống.





Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

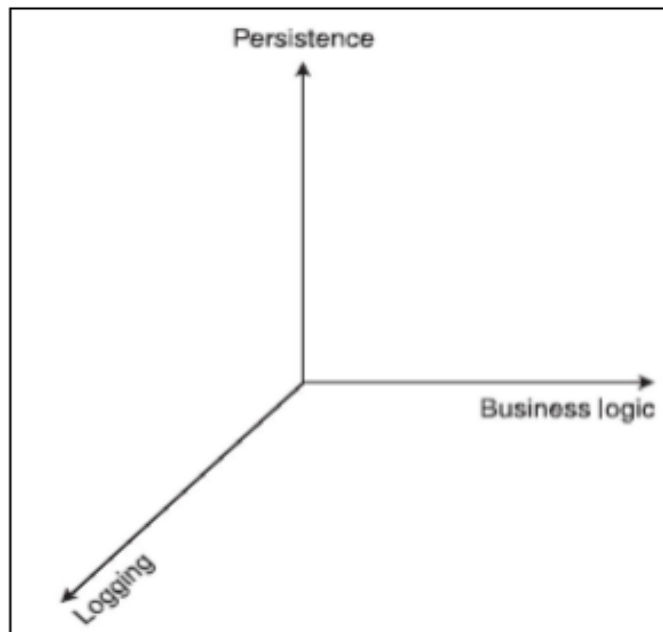
- Bước 1: Ánh xạ yêu cầu người dùng sử dụng mô hình AOP





Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

- Bước 2 +3: Thiết kế concern lõi và concern đan xen. Trong mô hình này, sự phụ thuộc giữa các module là mô hình đa chiều.

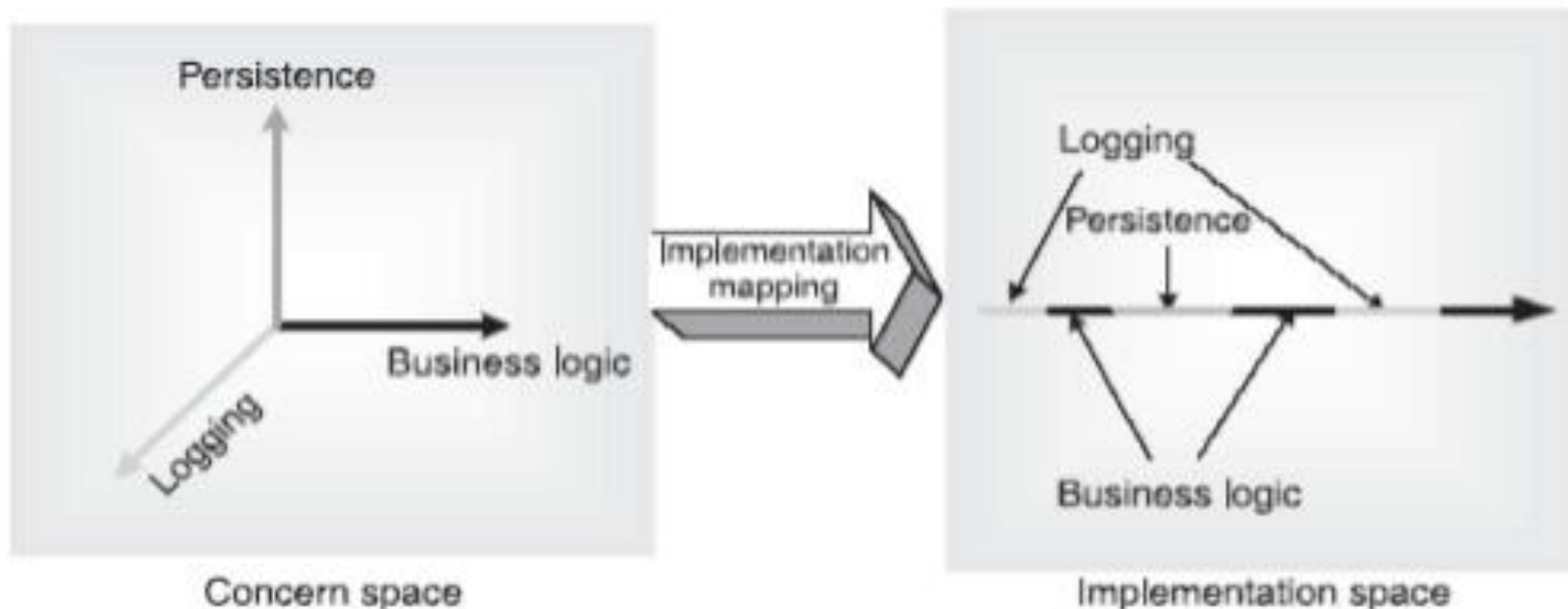


Mô hình một module yêu cầu người dùng bao gồm nhiều các concern



Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

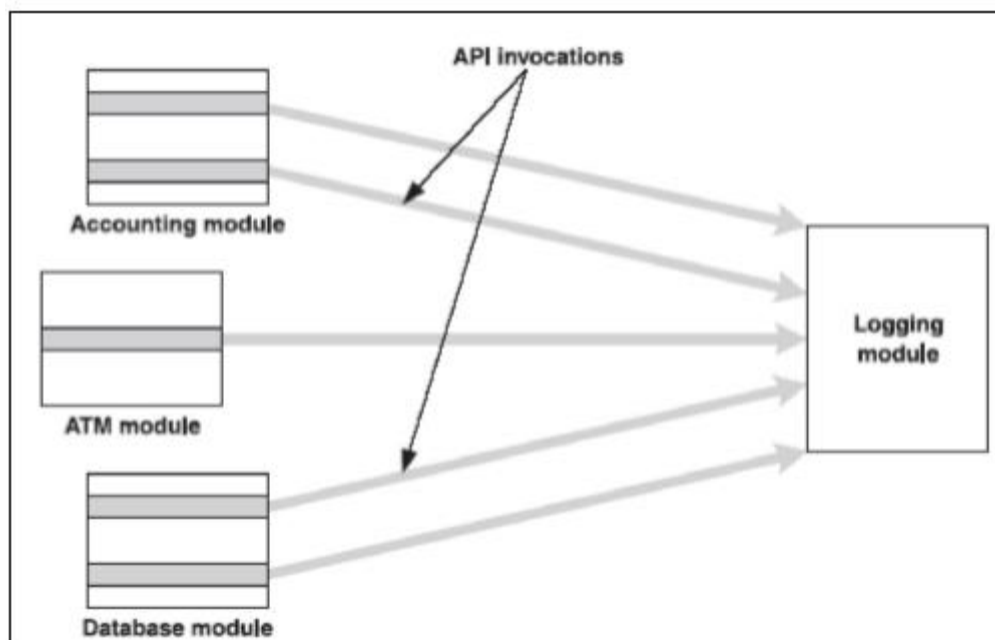
- Mô hình hệ thống biểu diễn nhiều chiều của các concern được ánh xạ sang các ngôn ngữ truyền thống một chiều





Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

- Ví dụ minh họa: Đây là mô hình thiết kế theo kiểu OOP truyền thống về bài toán trong ngành ngân hàng

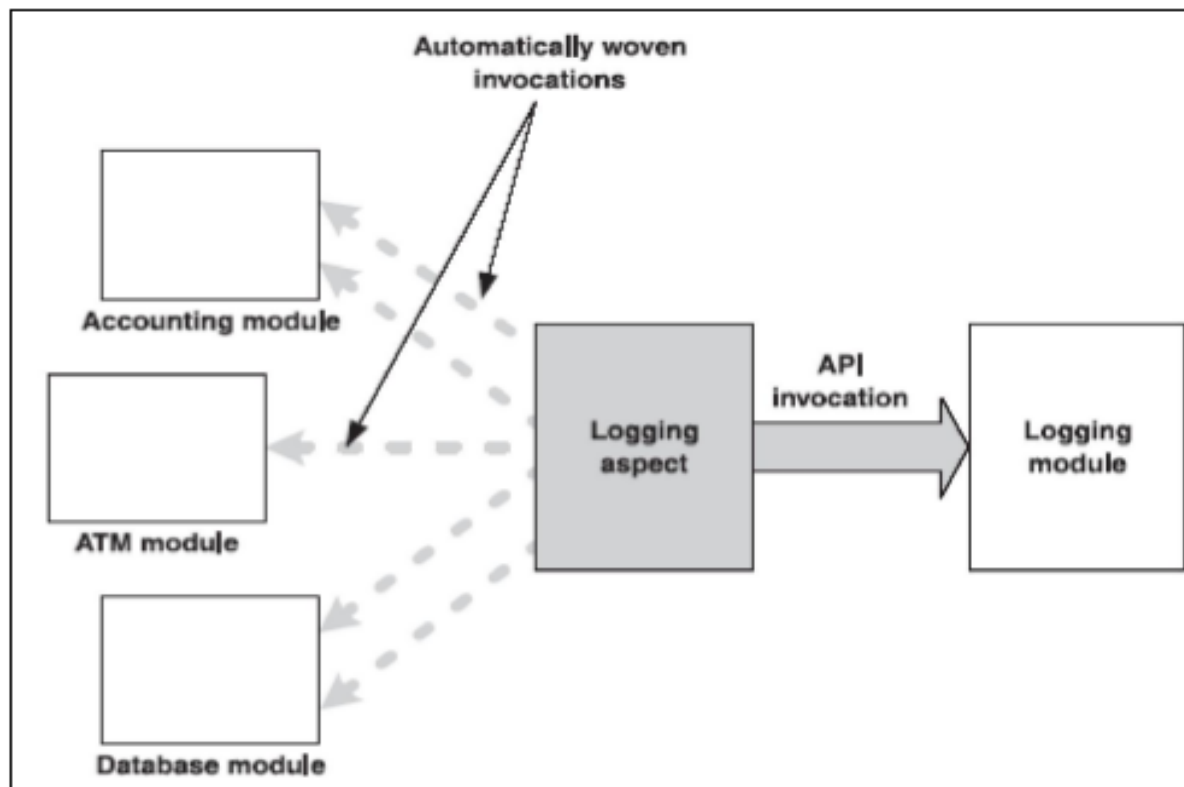


Các module yêu cầu logging đều phải nhúng các đoạn mã để gọi logging API



Thiết kế hệ thống tương quan hướng khía cạnh (Aspect-Oriented design)

- Đây là mô hình hệ thống tương quan được thiết kế theo AOP





Thảo luận

